

BookNest

AI-Powered Library Management System

Project Documentation

MERN Stack | MongoDB • Express.js • React • Node.js

1. Introduction

BookNest is a full-stack web application that digitises the day-to-day operations of a library. It replaces paper registers and spreadsheets with a single online system where members can discover books, reserve titles that are currently checked out and follow their own loans, while librarians and administrators handle issuing, returns, fines, memberships and analytics from a dedicated dashboard.

Beyond the basic catalogue and circulation features, BookNest adds three capabilities that make it stand out from a conventional library tool:

- **QR-based member cards.** every member gets a unique library card number (BN-xxxxxx) with a scannable QR code, so staff can identify a member at the desk in a second using the device camera.
- **AI damage inspection.** a photo is taken when a book is issued and another when it is returned; the system compares the two images, produces a damage score and heat-map, and proposes a damage fee automatically.
- **AI Book Advisor.** a chat drawer that recommends similar titles from the live catalogue, using a Hugging Face language model when an API key is configured and a built-in semantic matcher otherwise.

The platform is implemented with the MERN stack. The React single-page application communicates with a RESTful Express API secured by JSON Web Tokens, and all data is stored in MongoDB through Mongoose schemas. Scheduled jobs automatically flag overdue loans and calculate fines every night.

2. Scenario-Based Case Study

The following scenario shows how BookNest is used in practice by the three kinds of users it serves.

Meera, a Library Member

Meera is a college student who often finds the books she needs already checked out. She registers on BookNest with her name, email and password and immediately receives a library card number and QR card. While browsing the catalogue she filters by category and availability, opens a book page and sees how many copies are on the shelf and where the shelf is located. Because every copy of a data-structures textbook is out, she places a reservation. When a copy is returned, BookNest holds it for her for 48 hours. From the My Loans page she keeps track of due dates, any fines and the status of her reservations. When unsure what to read next, she opens the AI Book Advisor and asks for books similar to one she enjoyed.

Ravi, the Librarian

Ravi logs in to the staff dashboard, where the Overview tab shows total books, members, active loans, overdue loans and fines collected. When Meera arrives to collect her book, Ravi opens Check In / Out, scans the QR code on her card, picks the book and captures a baseline photo of its condition. The system checks that her membership is active and that she is under her borrowing limit before issuing the book with a due date. When the book comes back, Ravi photographs it again; the AI inspection compares both photos, highlights new marks and calculates a damage fee, and the return also adds any late fine. He can then mark the fine as paid or waived.

Alex, the Administrator

Alex has all of Ravi's abilities and can additionally change user roles. He reviews the analytics, suspends accounts that violate library rules, adjusts a member's borrowing limit and configures the Hugging Face API key and model names from the AI Settings tab.

3. System Requirements

The tools and machine specifications below are needed to develop, run and test BookNest.

3.1 Software Requirements

Software	Purpose
Windows 10/11, macOS or Linux	Cross-platform development and testing.
Node.js v20 or above	JavaScript runtime for the Express backend and the Vite build tools.
npm (bundled with Node.js)	Installs backend and frontend dependencies.
MongoDB (local Community Server or Atlas)	NoSQL database that stores users, books, loans and reservations.
React 19 + Vite	Frontend library and build tool for the single-page application.
Express 5 + Mongoose 9	REST API framework and MongoDB object modelling.
Tailwind CSS 4	Utility-first styling.
Google Chrome / Firefox (latest)	Rendering the UI and providing camera access for QR scanning and photos.
Postman (optional)	Testing API endpoints during development.
Visual Studio Code	Code editor with integrated terminal and Git support.
Hugging Face account (optional)	API token for the hosted vision and chat models.

3.2 Hardware Requirements

Component	Specification
Processor	Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or better
RAM	Minimum 8 GB (16 GB recommended) to run the servers, editor and browser together
Storage	At least 1 GB free for packages, MongoDB data and project files
Display	1366 x 768 or higher
Camera	Webcam or phone camera for QR scanning and book photos (a file-upload fallback is available)

4. Technical Architecture

BookNest follows a three-tier client-server architecture. The presentation tier is a React single-page application, the application tier is a Node.js/Express REST API and the data tier is MongoDB.

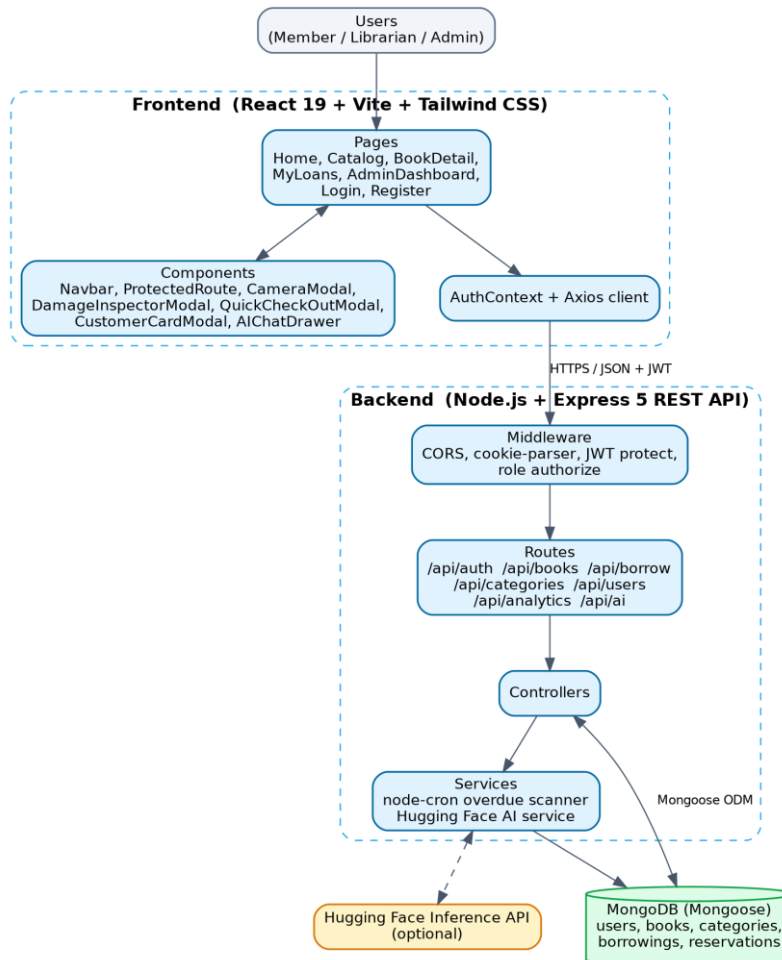


Figure 1: BookNest technical architecture

1. **User interaction.** A member, librarian or administrator opens the React application in the browser. React Router decides which page to render, and protected routes redirect anonymous visitors to the login page.
2. **API request.** The page calls the backend through a shared Axios client, which sends the JWT with every request (as an HTTP-only cookie and as a Bearer header).
3. **Security middleware.** Express parses the request, applies CORS and cookie handling, then the protect middleware verifies the token and loads the user, and authorize checks that the role is allowed to use the route.
4. **Business logic.** The matching controller validates the input, applies library rules (borrowing limits, fines, reservation queue) and reads or writes data through Mongoose models.
5. **Background and AI services.** A node-cron job runs at midnight to flag overdue loans and compute fines. The Hugging Face service performs damage assessment and book recommendations, falling back to built-in algorithms when no API key is available.
6. **Response.** The API returns JSON, which the frontend renders as tables, cards, modals and charts.

5. ER Diagram

The database has five collections. The diagram below shows their attributes and how they relate to one another.

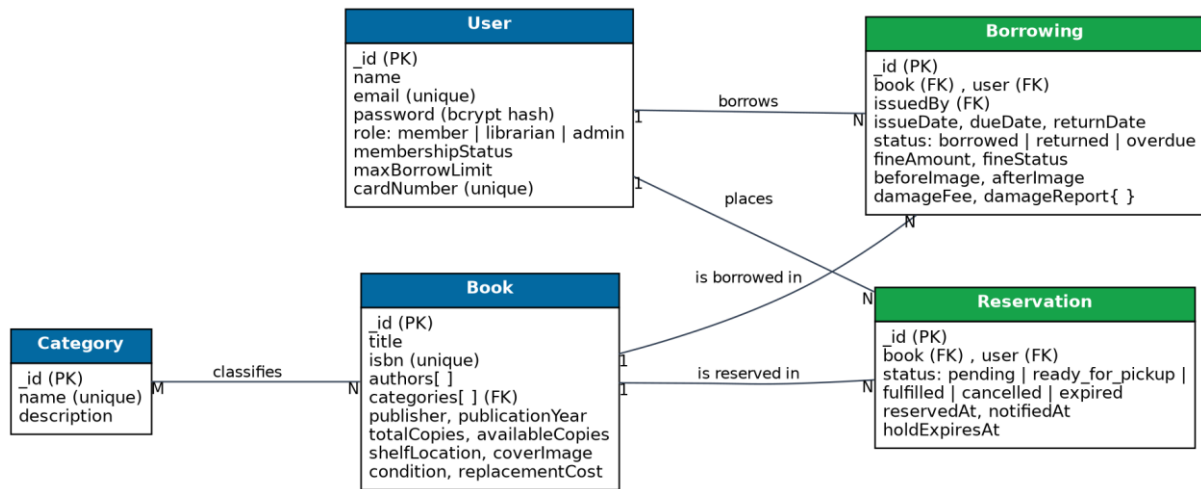


Figure 2: Entity-relationship diagram of BookNest

5.1 Entities

Entity	Description	Key attributes
User	Every person who can sign in: members, librarians and admins.	name, email, password (hashed), role, membershipStatus, maxBorrowLimit, cardNumber
Book	A title in the catalogue with its stock.	title, isbn, authors, categories, totalCopies, availableCopies, shelfLocation, coverImage, replacementCost
Category	A subject grouping such as Computer Science or Philosophy.	name, description
Borrowing	One loan of one book to one member, including fines and damage data.	issueDate, dueDate, returnDate, status, fineAmount, fineStatus, beforeImage, afterImage, damageFee, damageReport
Reservation	A member's place in the queue for a book that has no available copy.	status, reservedAt, notifiedAt, holdExpiresAt

5.2 Relationships

- **User – Borrowing (1 : N)**: a member can have many loans; each loan also records the staff member who issued it (issuedBy).
- **Book – Borrowing (1 : N)**: a book can be lent many times over its life.
- **User – Reservation (1 : N) and Book – Reservation (1 : N)**: many members can queue for the same book; a unique partial index stops one member holding two active reservations for the same title.
- **Category – Book (M : N)**: a book stores an array of category references, so it can belong to several categories.

5.3 Keys

Each collection uses MongoDB's `_id` ObjectId as its primary key. Foreign keys are ObjectId references (for example `Borrowing.book` references `Book` and `Borrowing.user` references `User`) and are resolved with `Mongoose populate()`. Unique indexes are placed on `User.email`, `User.cardNumber`, `Book.isbn` and `Category.name`.

6. Key Features

Feature	What it does
Registration and authentication	Members self-register; passwords are hashed with bcrypt; login issues a 7-day JWT stored in an HTTP-only cookie.
Catalogue search and filters	Search by title, author or ISBN; filter by category and availability; sort by title or year; paginated results.
Book details	Description, publisher, year, shelf location, condition and live copy count.
Reservations	Members queue for unavailable books. On return, the first person in the queue gets a 48-hour hold.
Issue and return	Staff issue books with a due date (14 days by default), enforcing membership status and borrowing limit.
QR member cards	Each member has a digital card with QR code; staff scan it with the camera or type a card number or email.
Automatic fines	A nightly job marks overdue loans and charges ₹5 per day; fines are also computed on return and can be paid or waived.
AI damage inspection	Before and after photos are compared pixel by pixel to produce a damage score, heat-map, defect list and fee.
AI Book Advisor	A chat drawer that suggests similar books from the live catalogue.
Admin dashboard	Tabs for overview analytics, check in/out, loans, books, damage inspection, members, holds and AI settings.
Role-based access control	Member, librarian and admin roles enforced on both API routes and front-end pages.

7. Roles and Responsibilities

7.1 Member

- **Account management:** register, log in and keep credentials secure.
- **Discovery:** search and browse the catalogue and view book details.
- **Reservations:** reserve books that are unavailable and cancel reservations no longer needed.
- **Loans:** return books on time, and monitor due dates, fines and history on the My Loans page.
- **AI advisor:** ask the chat assistant for reading suggestions.

7.2 Librarian

- Add, edit and delete books and categories, and update cover images with the camera.
- Issue and return books, scanning member QR cards and capturing before/after photos.
- Run AI damage inspections, record damage fees and mark fines as paid or waived.
- View all loans and reservations, search members and suspend or reactivate accounts, and change borrowing limits.
- Trigger the overdue scan manually and view dashboard analytics.

7.3 Administrator

- Everything a librarian can do.
- Change user roles (only admins may promote or demote users).
- Oversee system settings, including the Hugging Face API key and the vision and chat model names.

7.4 Permission Matrix

Capability	Guest	Member	Librarian	Admin
Browse catalogue, view books, use AI advisor	Yes	Yes	Yes	Yes
Register / log in	Yes	Yes	Yes	Yes
View own loans, reserve and cancel own reservation	—	Yes	Yes	Yes
Add / edit / delete books, add categories	—	—	Yes	Yes
Issue and return books, pay or waive fines	—	—	Yes	Yes
View all members, loans, reservations, analytics	—	—	Yes	Yes
Suspend members, change borrow limits	—	—	Yes	Yes
Change user roles	—	—	—	Yes

8. User Flow

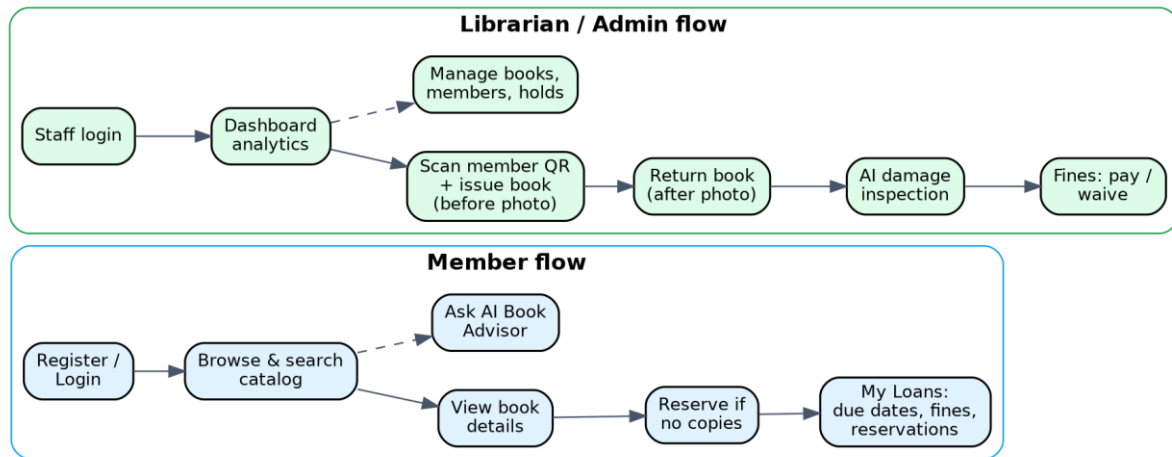


Figure 3: Member and staff workflows

8.1 Member Flow

- **Register / Login** – create an account or sign in securely.
- **Browse and search** – find books using filters and sorting.
- **View details and reserve** – check availability and queue for unavailable titles.
- **My Loans** – track due dates, fines and reservation status.
- **AI Book Advisor** – ask for similar books at any time.

8.2 Librarian / Admin Flow

- **Staff login** – authenticate and open the dashboard.
- **Issue** – scan the member card, choose a book and capture a baseline photo.
- **Return** – capture the after photo; late fines are calculated automatically.
- **Inspect and settle** – review the AI damage report, then pay or waive fees.
- **Manage** – maintain books, members and holds, and review analytics.

10. Project Setup and Configuration

10.1 Project Folder Structure

```

booknest/
├── backend/
│   ├── src/
│   │   ├── config/      db.js
│   │   ├── controllers/ auth, book, borrow, category, user, analytics, ai
│   │   ├── middlewares/ authMiddleware.js
│   │   ├── models/      user, Book, Category, borrowing, Reservation
│   │   ├── routes/      one router per controller
│   │   ├── services/     cronService.js, huggingFaceService.js
│   │   ├── utils/        generateToken.js
│   │   ├── seed.js       sample data loader
│   │   └── server.js     application entry point
│   ├── .env
│   └── package.json
└── frontend/
    ├── src/ (pages, components, context, api)
    ├── index.html
    ├── vite.config.js
    └── package.json

```

10.2 Backend Setup

1. Open the backend folder in the VS Code terminal.
2. Install the dependencies (express, mongoose, jsonwebtoken, bcryptjs, cookie-parser, cors, dotenv, node-cron, nodemon):

```

cd backend
npm install

```

3. Create a .env file in the backend folder (see the table below).
4. Load the sample data (this clears existing collections first):

```
node src/seed.js
```

5. Start the API server. It listens on port 5000 by default:

```

npm run dev      # development with nodemon
npm start        # production

```

10.3 Environment Variables

Variable	Required	Description
PORT	No	API port; defaults to 5000.
MONGO_URI	Yes	MongoDB connection string, for example mongodb://127.0.0.1:27017/booknest.
JWT_SECRET	Yes	Long random string used to sign tokens.
NODE_ENV	No	development or production; production turns on secure cookies.
HUGGINGFACE_API_KEY or HF_TOKEN	No	Enables the hosted Hugging Face models; without it the built-in engines are used.

Variable	Required	Description
HF_VISION_MODEL / HF_CHAT_MODEL	No	Override the default vision and chat model names.

10.4 Frontend Setup

1. Open the frontend folder in a second terminal.
2. Install packages and start the Vite dev server (<http://localhost:5173>):

```
cd frontend
npm install
npm run dev
```

3. Create a production build when needed:

```
npm run build
```

The Axios client in `src/api/axios.js` points to `http://localhost:5000/api`, and the backend CORS configuration accepts requests from the Vite development server.

10.5 Demo Accounts (created by the seed script)

Role	Email	Password
Admin	admin@booknest.com	password123
Librarian	librarian@booknest.com	password123
Member	sarah@example.com	password123

These accounts are for demonstration only and must be removed or changed before any real deployment.

11. Backend Development

11.1 Backend Structure

Controllers

File	Responsibility
authController.js	Register members, log in, return the current user and log out (clears the cookie).
bookController.js	Catalogue listing with search, filter, sort and pagination; create, update, delete books; update cover image.
borrowController.js	Issue and return books, my loans, reservations, fines, overdue scan, damage inspection on a loan.
categoryController.js	List and create categories (duplicate names are rejected, case-insensitive).
userController.js	List and filter users with loan and fine counts, update status, limit or role, look up a member by QR code, card number, id or email.
analyticsController.js	Dashboard KPIs, top five most borrowed books and category distribution using aggregation pipelines.
aiController.js	Damage assessment, AI chat, AI status and runtime configuration.

Middleware, Utilities and Services

- **authMiddleware.js** – protect reads the JWT from the cookie or the Authorization header, loads the user, blocks suspended accounts and assigns a card number to legacy users; `authorize(...roles)` restricts a route to specific roles.
- **generateToken.js** – signs a 7-day JWT and stores it in an HTTP-only, SameSite=Lax cookie (secure in production).
- **cronService.js** – schedules the overdue scanner with node-cron at 00:00 daily.
- **huggingFaceService.js** – damage assessment, image difference analysis and the similar-books recommender.

11.2 REST API Reference

Method	Endpoint	Access	Purpose
POST	/api/auth/register	Public	Create a member account
POST	/api/auth/login	Public	Authenticate and receive a token
GET	/api/auth/me	Private	Current user profile
POST	/api/auth/logout	Public	Clear the auth cookie
GET	/api/books	Public	List books (search, category, status, sortBy, page, limit)
GET	/api/books/:id	Public	Single book
POST	/api/books	Staff	Add a book
PUT	/api/books/:id	Staff	Update a book
PUT	/api/books/:id/image	Staff	Change the cover image
DELETE	/api/books/:id	Staff	Delete a book
GET	/api/categories	Public	List categories
POST	/api/categories	Staff	Add a category

Method	Endpoint	Access	Purpose
GET	/api/borrow/my-loans	Private	Own loans and reservations
POST	/api/borrow/reserve	Private	Reserve an unavailable book
DELETE	/api/borrow/reserve/:id	Private	Cancel a reservation
GET	/api/borrow/all	Staff	All loans (status, search)
GET	/api/borrow/reservations	Staff	All reservations
POST	/api/borrow/issue	Staff	Issue a book to a member
POST	/api/borrow/return/:id	Staff	Return a book and compute fees
PUT	/api/borrow/inspect/:id	Staff	Save damage inspection on a loan
POST	/api/borrow/pay-fine/:id	Staff	Mark a fine paid or waived
POST	/api/borrow/scan-overdue	Staff	Run the overdue scan now
GET	/api/users	Staff	List members with loan statistics
GET	/api/users/lookup/:code	Staff	Find a member by QR / card / id / email
PUT	/api/users/:id	Staff	Update status, limit (role: admin only)
GET	/api/analytics/dashboard	Staff	Dashboard metrics
POST	/api/ai/chat	Public	AI book advisor
GET	/api/ai/status	Public	AI engine status
POST	/api/ai/damage-assess	Staff	Run AI damage assessment
POST	/api/ai/config	Staff	Update AI key and model names
GET	/health	Public	Server health check

Staff = librarian or admin.

11.3 Key Business Rules

Rule	Implementation
Borrowing limit	A member cannot hold more active loans (borrowed or overdue) than <code>maxBorrowLimit</code> (default 3).
Loan period	14 days by default; staff may choose a different number of days at issue time.
Membership check	Books can only be issued to members whose <code>membershipStatus</code> is active; suspended users are blocked at login-protected routes.
Late fine	₹5 per late day, calculated by the nightly cron job and again when the book is returned.
Reservation queue	Reservations are served first come, first served. On return, the oldest pending reservation becomes <code>ready_for_pickup</code> with a 48-hour hold.
Stock tracking	<code>availableCopies</code> decreases on issue and increases on return, unless the copy goes to a waiting reservation.
Fine settlement	Fines and damage fees start as unpaid and can be marked paid or waived by staff.

12. Database Development

12.1 Configure MongoDB

- Install Mongoose in the backend project: `npm install mongoose`.
- Create the database connection (`src/config/db.js`).
- Create the schemas and models (`src/models`).

12.2 Database Connection

The connection is opened once when the server starts. If it fails, the process exits so that the API never runs without a database.

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI);
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`Database connection error: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;
```

12.3 Schemas and Models

The schemas are designed from the ER diagram in Section 5.

Model file	Collection	Purpose and notable details
user.js	users	Accounts. Email is unique and validated, password is hashed in a pre-save hook and hidden by default (<code>select: false</code>), and a card number BN-xxxxxx is generated automatically.
Book.js	books	Catalogue. Unique ISBN, category references, copy counters with minimum values, a text index on title, authors and description, and a virtual status field (<code>AVAILABLE / OUT_OF_STOCK</code>).
Category.js	categories	Unique category names with an optional description.
borrowing.js	borrowings	Loans. Status, fine fields, before/after images, an embedded <code>damageReport</code> , a <code>totalFees</code> virtual and indexes on user, book and due date.
Reservation.js	reservations	Hold queue. A partial unique index prevents duplicate active reservations per member and book.

Example – the User schema and password hook:

```
const userSchema = new mongoose.Schema({
  name: { type: String, required: [true, 'Name is required'], trim: true },
  email: { type: String, required: true, unique: true, lowercase: true, trim: true },
  password: { type: String, required: true, minlength: 6, select: false },
  role: { type: String, enum: ['member', 'librarian', 'admin'], default: 'member' },
  membershipStatus: { type: String, enum: ['active', 'suspended', 'expired'], default: 'active' },
});
```

```
maxBorrowLimit: { type: Number, default: 3 },
cardNumber: { type: String, unique: true, sparse: true, trim: true },
}, { timestamps: true });

userSchema.pre('save', async function () {
  if (!this.cardNumber) this.cardNumber = `BN-${Math.floor(100000 + Math.random() * 900000)}`;
  if (!this.isModified('password')) return;
  this.password = await bcrypt.hash(this.password, await bcrypt.genSalt(10));
});
```

13. Frontend Development

13.1 Frontend Structure

Pages (src/pages)

File	Route	Description
Home.jsx	/	Landing page with highlights and featured books.
Catalog.jsx	/books	Searchable, filterable, paginated catalogue; staff can add books and quick-issue from here.
BookDetail.jsx	/books/:id	Full book information; members reserve, staff issue, edit, delete or change the cover photo.
Login.jsx / Register.jsx	/login, /register	Authentication forms.
MyLoans.jsx	/my-loans	Member loans, fines, history and reservations (login required).
AdminDashboard.jsx	/admin/dashboard	Staff console with eight tabs (librarian or admin only).

Components (src/components)

File	Description
Navbar.jsx / Footer.jsx	Site-wide navigation with role-aware links, and footer.
ProtectedRoute.jsx	Redirects guests to login and shows an "Access Restricted" message for wrong roles.
BookCard.jsx	Catalogue card showing cover, title, authors and availability.
Modal.jsx / ErrorBoundary.jsx	Reusable dialog and a safety net for rendering errors.
CameraModal.jsx	Webcam capture with front/back camera switching and a file-upload fallback.
QuickCheckOutModal.jsx	Three-step issue wizard: scan or select member, choose book, confirm with baseline photo. Uses jsQR to read QR codes.
CustomerCardModal.jsx	Displays the digital library card with a QR code generated by the qrcode library.
DamageInspectorModal.jsx	Compares before and after photos on an HTML canvas and produces the damage score and heat-map.
AIChatDrawer.jsx	Slide-in chat for the AI Book Advisor.

State and API Layer

- **context/AuthContext.jsx** – holds the logged-in user and exposes login, register and logout; restores the session on page load through /auth/me.
- **api/axios.js** – a preconfigured Axios instance with credentials enabled and an interceptor that adds the Bearer token.

13.2 Admin Dashboard Tabs

Tab	Purpose
Overview	KPI cards, popular books and category distribution, with shortcuts to the AI damage station and rapid check in/out.
Check In / Out	Scan a member, issue books and process returns.

Tab	Purpose
Loans	All loans with status filters, search and fine actions.
Books	Manage catalogue records.
Damage Inspection	Book damage inspection station with history of reports.
Members	Search members, view loan and fine counts, suspend or reactivate, change limits and roles.
Holds	View reservations and their status.
AI Settings	Configure the Hugging Face key and model names.

14. AI Modules

14.1 AI Damage Inspection

When a book is issued, staff capture a baseline photo. On return, a second photo is taken and both images are compared to decide whether the book has been damaged.

1. **Client-side analysis.** DamageInspectorModal draws both photos on a canvas, normalises brightness to cope with different lighting, searches a small neighbourhood around each pixel to tolerate camera-angle changes, and filters out isolated noise so that only connected clusters of change (stains, tears, scratches) remain. It also renders a difference heat-map.
2. **Server-side scoring.** The score is sent to POST /api/ai/damage-assess, where huggingFaceService maps it to a damage level and fee. If the client did not provide a score, the server computes a byte-level difference between the two images as a fallback. When a Hugging Face token is present, a vision model is also queried and its top predictions are attached to the report.
3. **Recording.** The report (score, level, defects, notes, fee, model used, timestamp) is saved inside the Borrowing document and adds an unpaid fee when damage is found.

Damage score	Level	Damage fee
0 – 5	None	₹0
6 – 32	Minor	₹50
33 – 65	Moderate	₹150
Above 65	Severe	The greater of ₹300 and the book's replacement cost

14.2 AI Book Advisor

The advisor loads the catalogue from MongoDB and scores every book against the user's message, using title, author, category and description keyword matches plus genre hints (for example "programming" boosts Computer Science). The top matches become the context for the answer. If a Hugging Face key is configured, an instruction-tuned chat model (Mistral-7B-Instruct by default) writes a friendly reply using that context; otherwise the built-in engine formats the recommendations, including copies available and shelf location. Because of this fallback, the feature keeps working without an API key or when the external service is unreachable.

15. Security Considerations

- **Password protection:** passwords are salted and hashed with bcrypt and never returned by default queries.
- **Token handling:** JWTs expire after 7 days and are sent as HTTP-only cookies; the front end also sends the token as a Bearer header.
- **Authorisation:** every staff endpoint uses protect followed by authorize, so the role is checked on the server regardless of what the UI shows.
- **Account suspension:** suspended members are rejected by the protect middleware on every request.
- **Data integrity:** unique indexes on email, ISBN, card number and category name, and validation on stock counters, prevent inconsistent records.
- **Secrets:** the database URI, JWT secret and API keys are kept in the .env file, which is excluded from version control by .gitignore.

16. Output Screenshots

Run the application and replace each placeholder below with a screenshot of the corresponding page.

1. Home Page

[Insert screenshot here]
/

2. User Registration Page

[Insert screenshot here]
/register

3. User Login Page

[Insert screenshot here]
/login

4. Catalogue Page

[Insert screenshot here]
/books – search, category filter, availability filter

5. Book Detail Page

[Insert screenshot here]
/books/:id

6. My Loans Page (Member)

[Insert screenshot here]
/my-loans – loans, fines, reservations

7. AI Book Advisor

[Insert screenshot here]
Chat drawer opened from the floating button

8. Admin Dashboard – Overview

[Insert screenshot here]
/admin/dashboard

9. Quick Check Out – QR Scan

[Insert screenshot here]
Member scan step

10. Digital Library Card

[Insert screenshot here]
Customer card with QR code

11. Damage Inspection with Heat-map

[Insert screenshot here]
Before/after photos and AI report

12. Loans Management

[Insert screenshot here]
Dashboard – Loans tab

13. Members Management

[Insert screenshot here]
Dashboard – Members tab

14. Holds / Reservations

[**Insert** **screenshot** **here**]
Dashboard – Holds tab

15. AI Settings

[**Insert** **screenshot** **here**]
Dashboard – AI Settings tab

17. Conclusion and Future Enhancements

BookNest brings cataloguing, circulation, reservations, fines and analytics together in one secure web application, and adds QR identification and AI-assisted damage inspection to reduce manual effort and disputes at the returns desk. The layered MVC backend and component-based React frontend make it easy to maintain and extend.

Possible future enhancements include:

- Email or SMS notifications for due dates and reservation pick-up.
- Online payment of fines.
- Automatic expiry of reservation holds once the 48-hour window ends.
- Storing images in cloud file storage instead of inside the database documents.
- Automated tests, request rate limiting and stricter CORS rules for deployment.
- A mobile application for members.

Project Links

Item	Link
Demo video	https://drive.google.com/file/d/1dg6k1h7Kl9zIrl_92A5zRMsORsBlzaZ9/view?usp=sharing
Source code	https://github.com/kanupriyan-dotcom/BookNest